



Mobile Developer Technical Assessment Guide

This assessment is designed to evaluate your technical proficiency, system architecture design skills, and your ability to leverage AI tools to enhance productivity and problem-solving. Please develop a mobile application that demonstrates your best work within the given technical constraints.

1. Task Overview

- **Topic:** Open (Develop a standalone app service using any Open API or custom data).
- **Duration:** 48 hours from the start of the task.
- **Submission:** Create a **Private GitHub Repository**, grant access to sungchan.bong@qcells.com, jason.jung02@qcells.com, adam.macdonald@qcells.com, and april.malovani@qcells.com as collaborators and share the repository URL.

2. Technical Stack & Requirements

[Android Candidates]

- **UI Framework:** Jetpack Compose (Required)
- **Architecture:** Multi-Module Architecture (Required)
- **Language:** Kotlin
- **Recommendations:** Coroutines/Flow, Dependency Injection (Hilt/Koin), and MVVM or MVI patterns.

[iOS Candidates]

- **UI Framework:** SwiftUI (Required)
- **Architecture:** Multi-Module Architecture (Required)
- **Language:** Swift

- **Recommendations:** Swift Concurrency (Async/Await), Combine or TCA/MVVM patterns, and SPM (Swift Package Manager).

3. Core Functional Requirements

Regardless of the app's theme, the following **four core features** must be implemented:

1. **Data List Interaction:** Fetch data from an external API or local source with list paging and navigation to a detailed view.
2. **Search & Filtering:** A functional real-time search bar or a multi-criteria data filtering system.
3. **Data Persistence & State Management:** Implementation of "Favorites," local caching, or user preference storage.
4. **Network & Error Handling:** Graceful handling of offline states, API errors, and user-facing notifications (Toasts, Alerts, etc.).

4. AI Utilization Guidelines

The use of **AI tools (ChatGPT, Claude, GitHub Copilot, etc.) is highly encouraged**. To evaluate how you integrate these tools into your professional workflow, you must include an **"AI Documentation"** section in your **README.md** detailing:

- **Scope of Usage:** Where did you use AI? (e.g., Boilerplate modules, complex algorithm logic, Unit Test generation).
- **Prompt Examples:** Provide 1–2 examples of effective prompts or complex problem-solving dialogues.
- **Verification Process:** Explain how you validated the AI-generated code for errors, security, or performance.
- **Reflection:** A brief note on how AI impacted your productivity or the quality of the final architecture.

5. Evaluation Criteria

1. **Architecture (40%):** Effective modularization, Separation of Concerns (SoC), scalability, and ease of maintenance.
2. **Code Quality (30%):** Readability, consistent coding conventions, robust error handling, and resource optimization.

3. **AI Workflow Literacy (20%):** Logical process in defining and solving problems using AI tools.
4. **Polish & Stability (10%):** Attention to UI/UX detail and overall application stability.

6. Submission Checklist

- **GitHub Repository:**
 - Set the repository to **Private**.
 - Push the final code to the main branch.
- Grant access to sungchan.bong@qcells.com, jason.jung02@qcells.com, adam.macdonald@qcells.com, and april.malovani@qcells.com as collaborators and share the repository URL.
- **README.md File:**
 - Project introduction and setup instructions (include steps for API keys if necessary).
 - List of libraries used and an architectural diagram/description.
 - **Required:** The AI Utilization Guide (as specified in Section 4).